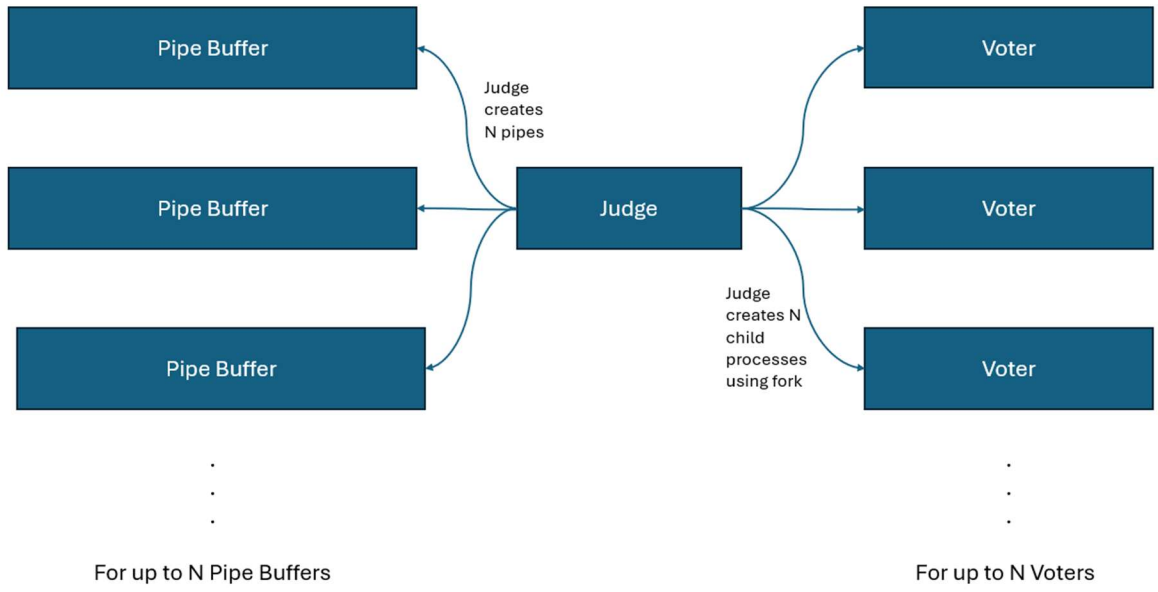
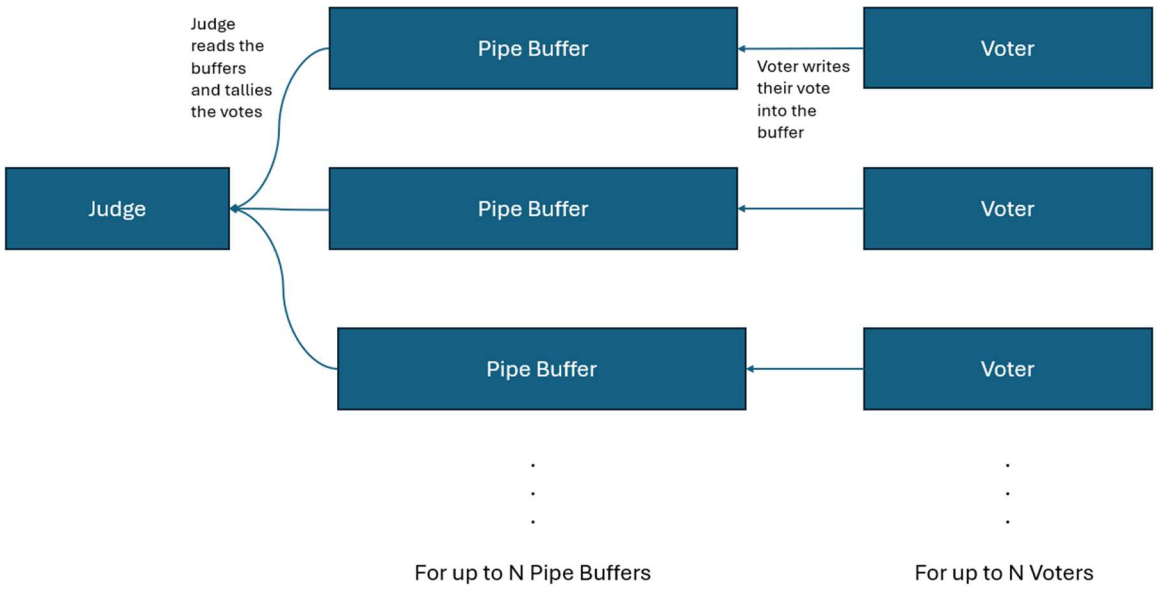
Diagram describing how processes interact/communicate with each other

Visualization of how the judge process creates the pipes and forks voter processes. (Steps 2 & 3 in the assignment instructions.)



Visualization of how the voters write their vote to the pipes and then the judge reads the pipes and tallies the votes.



To Compile:

gcc Assignment6 -o Assignment6

Note: I completed this assignment with gcc version gcc Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0

To Run:

./Assignment6

Program Running

Using parameters given in class example

```
kmfirth23@Kennedy-Laptop:~/CS4420/CS-4200-Assignment6$ ./Assignment6
Enter number of voters: 3
Enter max number of rounds: 2
------ Round 1 -------
--- Round 1: Voter 0 votes: YES.
--- Round 1: Voter 1 votes: YES.
--- Round 1: Voter 2 votes: NO.
--- Round 1: The voting result is a YES.
------ Round 2 -------
--- Round 2: Voter 0 votes: NO.
--- Round 2: Voter 1 votes: NO.
--- Round 2: Voter 2 votes: YES.
--- Round 2: The voting result is a NO.
```

If you do not enter enough voters

```
kmfirth23@Kennedy-Laptop:~/CS4420/CS-4200-Assignment6$ ./Assignment6
Enter number of voters: 2
Number of voters should be at least 3
```

Situation where a tie occurs

```
kmfirth23@Kennedy-Laptop:~/CS4420/CS-4200-Assignment6$ ./Assignment6
Enter number of voters: 4
Enter max number of rounds: 3
------ Round 1 -------
--- Round 1: Voter 0 votes: YES.
--- Round 1: Voter 1 votes: NO.
--- Round 1: Voter 2 votes: NO.
--- Round 1: Voter 3 votes: YES.
--- Round 1: The voting result is a TIE.
------ Round 2 -------
--- Round 2: Voter 0 votes: NO.
--- Round 2: Voter 1 votes: YES.
--- Round 2: Voter 2 votes: YES.
--- Round 2: Voter 3 votes: NO.
--- Round 2: The voting result is a TIE.
------ Round 3 -------
--- Round 3: Voter 0 votes: NO.
--- Round 3: Voter 1 votes: NO.
--- Round 3: Voter 2 votes: YES.
--- Round 3: Voter 3 votes: NO.
--- Round 3: The voting result is a NO.
```